Vehicle Basics

Conducción con físicas en Unity

Eva Gallardo Romero

26 de marzo de 2025

Índice

Enlace al repositorio de GitHub **2**

1 Descripción general **2**

2 Diagrama de clases **2**

3 Diseño **3**
3.1 Controles 3

4 Requisitos del sistema **3**
4.1 Requisitos funcionales 3
4.2 Requisitos no funcionales 3

5 Documentación técnica **4**
5.1 Movimiento.cs 4
5.2 CamaraFollow.cs 5
5.3 CollectableSpawner.cs y CollectibleBehaviour.cs 6
5.4 Obstacles.cs 6
5.5 GameEvents.cs y ScoreDisplayAnimator.cs 7

Enlace al repositorio de GitHub

Puedes consultar el proyecto completo en el siguiente enlace: Vehicle-basics.

La documentación del código generada con Doxygen está en evagr11.github.io/Vehicle-basics.

1 Descripción general

Vehicle Basics es un pequeño juego en **3D** hecho en **Unity** con **C#** en la asignatura de Programación de 1.º de DAM. Se controla una **mariquita** por un circuito cerrado: hay que recoger las **flores** que van apareciendo y esquivar los **obstáculos** que suben y bajan. Cada flor suma 10 puntos y, si chocas, pierdes 2 y el nivel vuelve a empezar.

Todo el movimiento está programado a mano con el **Rigidbody**, sin `WheelCollider`: aceleración hasta una velocidad máxima, giro con `AddTorque`, rozamiento que frena poco a poco al soltar el acelerador, freno, marcha atrás y un **freno de mano** que frena en seco, gira más rápido y deja **marcas de derrape**.

2 Diagrama de clases

`Movimiento` lleva la conducción y `CamaraFollow` la cámara. Las flores las crea `CollectableSpawner` y, al recogerlas, piden una nueva. Las flores y los obstáculos avisan con los eventos estáticos de `GameEvents`, que escucha el marcador.

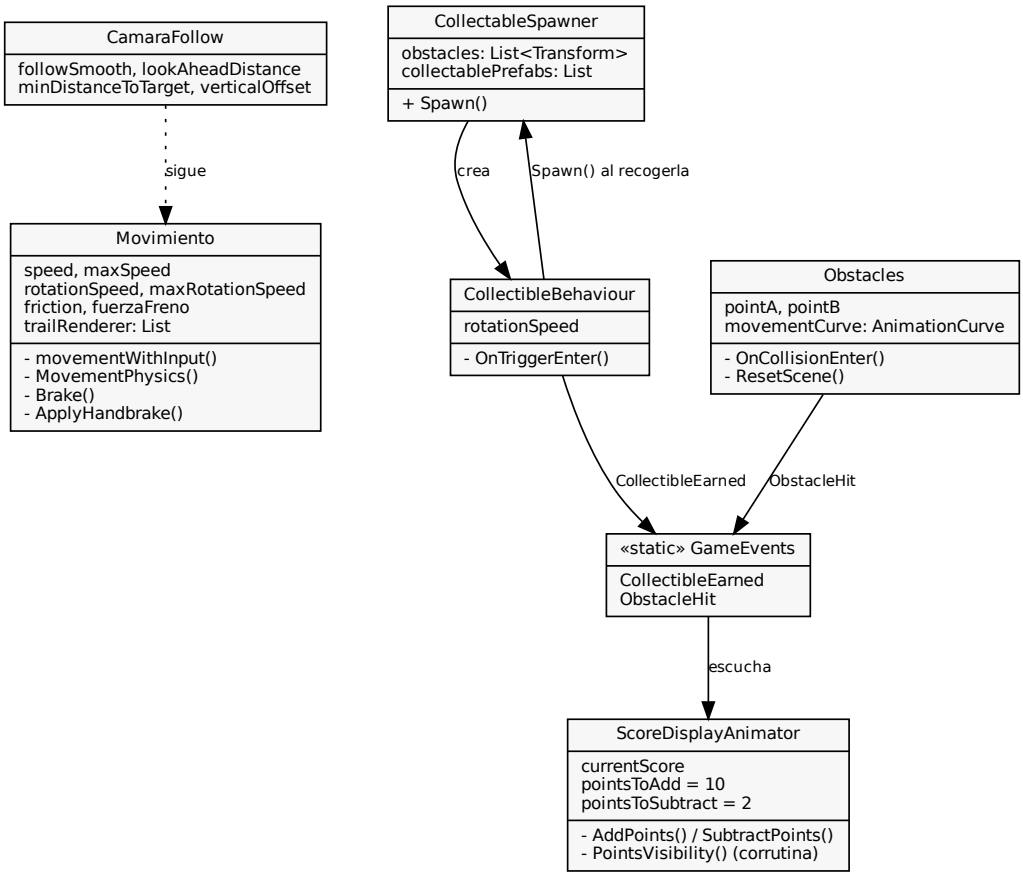


Figura 1: Diagrama de clases de Vehicle Basics

3 Diseño

La cámara va detrás de la mariquita, un poco por encima, y mira por delante de ella para que se vea la curva siguiente. Las flores giran sobre sí mismas para llamar la atención. El marcador está oculto y solo aparece con un fundido cuando se ganan o se pierden puntos, y vuelve a desaparecer a los dos segundos.

3.1 Controles

- **W**: acelerar.
- **S**: frenar y, parado, marcha atrás.
- **A / D**: girar.
- **Espacio**: freno de mano (derrape).

4 Requisitos del sistema

4.1 Requisitos funcionales

ID	Requisito funcional
RF01	El vehículo debe acelerar con **W** hasta una velocidad máxima y frenar poco a poco por rozamiento al soltarla.
RF02	Con **S** debe frenar si va hacia delante y dar marcha atrás si no.
RF03	Debe girar con **A/D**, con una velocidad de giro máxima.
RF04	El **freno de mano** debe frenar más fuerte, girar más y activar las marcas de derrape.
RF05	Las flores deben aparecer en uno de los puntos del circuito al azar, **sin repetir** el anterior.
RF06	Al recoger una flor se deben sumar **10 puntos** y crear otra.
RF07	Los obstáculos deben subir y bajar entre dos puntos siguiendo una curva de animación.
RF08	Al chocar con un obstáculo se deben restar **2 puntos** (sin bajar de 0) y reiniciar el nivel al segundo.
RF09	El marcador debe aparecer y desaparecer con un **fundido** al cambiar los puntos.

4.2 Requisitos no funcionales

ID	Requisito no funcional
RNF01	El juego debe estar hecho en **Unity 2023.2** con scripts en **C#**.
RNF02	El movimiento debe hacerse con **Rigidbody**: la lectura del teclado en `Update` y las físicas en `FixedUpdate`.
RNF03	Los objetos se comunican con **UnityEvent** estáticos; los que escuchan se quitan al desactivarse.
RNF04	Los valores de conducción, de los obstáculos y del marcador se configuran desde el Inspector.
RNF05	Los scripts deben tener comentarios de documentación `///` para Doxygen.

5 Documentación técnica

Este anexo agrupa la documentación por scripts, separándola en la **ubicación** donde se encuentra, una **descripción general** y las **funciones** explicadas de manera individual con su **código** adjunto y comentado. Se omiten los `using` y los comentarios `///`.

5.1 Movimiento.cs

5.1.1 Ubicación

`Assets/Scripts/Movimiento.cs`

Descripción general:
La conducción de la mariquita.

5.1.2 Funciones

movementWithInput **Descripción:** Lee el teclado en cada fotograma y lo guarda para la física.

Código comentado:

```
void movementWithInput()
{
    rotationInput = 0;
    accelerating = Input.GetKey(KeyCode.W);
    deccelerating = Input.GetKey(KeyCode.S);
    handbrake = Input.GetKey(KeyCode.Space);
    if (Input.GetKey(KeyCode.A))
        rotationInput = -1;
    if (Input.GetKey(KeyCode.D))
        rotationInput = 1;
}
```

MovementPhysics **Descripción:** Gira, acelera o aplica rozamiento, frena o da marcha atrás, y limita la velocidad lineal y la de giro.

Código comentado:

```
void MovementPhysics()
{
    rb.AddTorque(rotationInput * rotationSpeed * transform.up * Time.fixedDeltaTime);
    // velocidad vista desde el vehículo: z positiva = hacia delante
    Vector3 localVelocity = transform.InverseTransformDirection(rb.velocity);

    if (accelerating)
    {
        if (localVelocity.z < maxSpeed)
            rb.velocity += speed * transform.forward * Time.fixedDeltaTime;
    }
    else
    {
        // rozamiento: reduce la velocidad poco a poco
        rb.velocity = new Vector3(
            rb.velocity.x * (1 / (1 + friction * Time.fixedDeltaTime)),
            0,
            rb.velocity.z * (1 / (1 + friction * Time.fixedDeltaTime)));
    }

    if (deccelerating)
    {
        if (localVelocity.z > 0)
            Brake();                                        // va hacia delante: frena
        else
            rb.velocity -= speed * transform.forward * Time.fixedDeltaTime;   //
                marcha atrás
    }

```

```
29      if (handbrake)
30          ApplyHandbrake();
31      else
32          foreach (TrailRenderer trail in trailRenderer)
33              trail.emitting = false;
34
35      if (rb.velocity.magnitude > maxSpeed)
36          rb.velocity = rb.velocity.normalized * maxSpeed;
37      if (rb.angularVelocity.magnitude > maxRotationSpeed)
38          rb.angularVelocity = rb.angularVelocity.normalized * maxRotationSpeed;
39  }
```

ApplyHandbrake **Descripción:** Frena con más fuerza, añade giro si se está girando y enciende las marcas de derrape.

Código comentado:

```
1  void ApplyHandbrake()
2  {
3      float handbrakeFuerza = 15f;
4      float extraRotation = 2f;
5      rb.velocity = new Vector3(
6          rb.velocity.x * (1 / (1 + handbrakeFuerza * Time.fixedDeltaTime)),
7          0,
8          rb.velocity.z * (1 / (1 + handbrakeFuerza * Time.fixedDeltaTime)));
9
10     if (rotationInput != 0)
11         rb.AddTorque(rotationInput * extraRotation * transform.up);
12
13     foreach (TrailRenderer trail in trailRenderer)
14         trail.emitting = true;                  // marcas en el suelo
15 }
```

5.2 CamaraFollow.cs

5.2.1 Ubicación

`Assets/Scripts/CamaraFollow.cs`

Descripción general:
Cámara en tercera persona.

5.2.2 Funciones

FixedUpdate **Descripción:** Mira por delante del jugador, se coloca detrás y por encima con `SmoothDamp` y nunca se acerca más de la distancia mínima.

Código comentado:

```
1  private void FixedUpdate()
2  {
3      transform.LookAt(player.transform.position + lookAheadDistance * player.transform.
           forward);
4      Vector3 targetPosition = player.position + minDistanceToTarget * -player.transform
           .forward
5                               + verticalOffset * Vector3.up;
6      transform.position = Vector3.SmoothDamp(transform.position, targetPosition,
7                                              ref currentVelocity, followSmooth);
8
9      Vector3 directionToPlayer = player.position - transform.position;
10     if (directionToPlayer.magnitude < minDistanceToTarget)
11         transform.position = player.position - directionToPlayer.normalized *
               minDistanceToTarget;
12 }
```

5.3 CollectableSpawner.cs y CollectibleBehaviour.cs

5.3.1 Ubicación

`Assets/Scripts`

Descripción general:
Las flores: dónde aparecen y qué pasa al recogerlas.

5.3.2 Funciones

Spawn **Descripción:** Elige un punto del circuito distinto del anterior y una flor al azar.
Código comentado:

```
public void Spawn()
{
    int newIndex;
    do
    {
        newIndex = Random.Range(0, obstacles.Count);
    } while (newIndex == lastObstacleIndex && obstacles.Count > 1);   // no repite
    lastObstacleIndex = newIndex;

    GameObject selectedPrefab = collectablePrefabs[Random.Range(0, collectablePrefabs.
        Count)];
    Vector3 spawnPosition = obstacles[newIndex].position;
    spawnPosition.y = 1f;
    Instantiate(selectedPrefab, spawnPosition, Quaternion.identity);
}
```

OnTriggerEnter **Descripción:** Al tocarla el jugador, avisa, pide otra flor y se destruye.
Código comentado:

```
private void OnTriggerEnter(Collider other)
{
    if (other.CompareTag("Player"))
    {
        GameEvents.CollectibleEarned.Invoke();
        CollectableSpawner spawner = Object.FindFirstObjectByType<CollectableSpawner
            >();
        if (spawner != null)
            spawner.Spawn();
        Destroy(gameObject);
    }
}
```

5.4 Obstacles.cs

5.4.1 Ubicación

`Assets/Scripts/Obstacles.cs`

Descripción general:
Obstáculos que suben y bajan y reinician el nivel al chocar.

5.4.2 Funciones

Update **Descripción:** Avanza el tiempo, evalúa la curva y mueve el obstáculo entre A y B, cambiando de sentido al llegar.
Código comentado:

```
elapsedTime += Time.deltaTime * speed;
float distance = pointB.position.y - pointA.position.y;
float t = elapsedTime / distance;
```

```
4  float curveT = movementCurve.Evaluate(t);          // la curva marca el ritmo
5
6  if (goingUp)
7  {
8      transform.position = new Vector3(transform.position.x, pointA.position.y + curveT
           * distance, transform.position.z);
9      if (transform.position.y >= pointB.position.y)
10     {
11         goingUp = false;
12         elapsedTime = 0f;
13     }
14 }
15 else
16 {
17     // lo mismo bajando de B a A
18 }
```

OnCollisionEnter **Descripción:** Resta puntos una sola vez y recarga la escena al cabo de un segundo.

Código comentado:

```
1  private void OnCollisionEnter(Collision collision)
2  {
3      if (!hitPlayer && collision.gameObject.CompareTag("Player"))
4      {
5          hitPlayer = true;                    // evita restar varias veces
6          GameEvents.ObstacleHit.Invoke();
7          Invoke(nameof(ResetScene), 1f);
8      }
9  }
```

5.5 GameEvents.cs y ScoreDisplayAnimator.cs

5.5.1 Ubicación

`Assets/Scripts`

Descripción general:
Eventos del juego y marcador animado.

5.5.2 Funciones

OnEnable y OnDisable **Descripción:** El marcador se suscribe a los eventos y se da de baja al desactivarse, porque los eventos estáticos sobreviven al recargar la escena.

Código comentado:

```
1  public static class GameEvents
2  {
3      public static UnityEvent CollectibleEarned = new UnityEvent();
4      public static UnityEvent ObstacleHit = new UnityEvent();
5  }
6
7  void OnEnable()
8  {
9      GameEvents.CollectibleEarned.AddListener(AddPoints);
10     GameEvents.ObstacleHit.AddListener(SubtractPoints);
11 }
12
13 void OnDisable()
14 {
15     GameEvents.CollectibleEarned.RemoveListener(AddPoints);
16     GameEvents.ObstacleHit.RemoveListener(SubtractPoints);
17 }
```

SubtractPoints y TriggerPointsAnimation **Descripción:** Resta sin bajar de 0 y relanza la animación del marcador, parando la anterior si seguía en marcha.

Código comentado:

```
void SubtractPoints()
{
    currentScore -= pointsToSubtract;
    if (currentScore < 0) currentScore = 0;
    UpdatePointsText();
    TriggerPointsAnimation();
}

void TriggerPointsAnimation()
{
    if (animationCoroutine != null)
        StopCoroutine(animationCoroutine);
    animationCoroutine = StartCoroutine(PointsVisibility());   // fundido de entrada y
        salida
}
```